

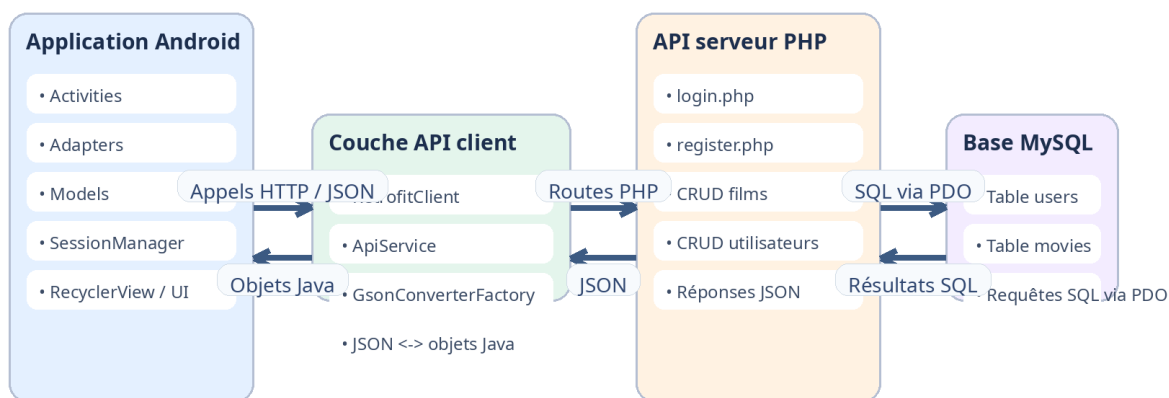
Rapport d'étude du projet MovieApp

Analyse du code source Android Studio, de l'API PHP et de la connexion avec la base de données

Type de projet	Application Android de gestion d'un catalogue de films
Technologies	Java, Android SDK, Retrofit, Gson, PHP, PDO, MySQL
Fonctions principales	Connexion, inscription, consultation des films, gestion des films, gestion des utilisateurs
Documents livrés	Version Word et version PDF du rapport

Remarque : ce rapport est rédigé dans un style d'analyse de projet. Il décrit le fonctionnement réel observé dans les fichiers du projet fournis.

Architecture générale du projet



Idée clé : l'application Android ne contacte pas directement la base de données. Elle envoie des requêtes à une API PHP (via Retrofit), puis l'API utilise PDO pour lire/écrire dans MySQL et renvoie une réponse JSON à l'application.

Figure 1 — Schéma général du fonctionnement du projet.

1. Introduction

MovieApp est une application Android développée en Java. Son objectif est de permettre à un utilisateur de consulter une liste de films, d'afficher le détail d'un film et, pour un administrateur, de gérer le contenu de l'application. Le projet repose sur une organisation assez classique : le téléphone affiche les écrans, une API PHP traite les demandes, puis une base MySQL stocke les données.

L'intérêt de ce projet est qu'il montre un exemple simple de communication entre une application mobile et un serveur distant. On retrouve donc les éléments essentiels d'un petit système d'information : authentification, affichage de données, ajout, modification et suppression.

2. Présentation générale du projet

D'après les fichiers analysés, le projet distingue deux profils d'utilisation. Le profil utilisateur permet surtout la consultation du catalogue. Le profil administrateur dispose d'un tableau de bord et d'actions supplémentaires pour ajouter des films, modifier les fiches existantes et gérer les comptes.

L'application Android ne se connecte pas directement à la base de données. Elle communique avec un ensemble de scripts PHP placés dans le dossier `php_api`. Ces scripts reçoivent les requêtes HTTP, exécutent les requêtes SQL nécessaires via PDO puis renvoient une réponse au format JSON.

3. Organisation du code

Le code Android est organisé de façon simple et lisible. Le dossier principal contient les activités, les adaptateurs, les modèles et une classe utilitaire pour la session.

Dossier / fichier	Rôle dans le projet
activities	Contient les écrans principaux comme LoginActivity, UserHomeActivity, AddMovieActivity ou ManageUsersActivity.
adapters	Contient MovieAdapter et UserAdapter pour l'affichage des listes dans RecyclerView.
models	Regroupe les classes Movie, User et les objets de réponse JSON comme LoginResponse et MoviesResponse.
api/RetrofitClient.java	Définit l'adresse du serveur et crée l'instance Retrofit utilisée par l'application.
api/ApiService.java	Déclare les routes accessibles depuis Android.
utils/SessionManager.java	Gère la session locale avec SharedPreferences.

php_api	Contient les scripts serveur qui effectuent les opérations sur la base MySQL.
---------	---

4. Partie Android

La partie Android correspond à l'interface visible par l'utilisateur. Chaque activité représente un écran précis et chaque écran envoie éventuellement une requête au serveur.

4.1 Les écrans principaux

- LoginActivity : écran de connexion. Il récupère le nom d'utilisateur et le mot de passe puis appelle l'API de connexion.
- IdentificationActivity : écran d'inscription. Il vérifie les champs saisis avant d'appeler register.php.
- UserHomeActivity : affiche la liste des films, la recherche et les filtres par genre.
- MovieDetailActivity : affiche le détail d'un film à partir de son identifiant.
- AdminDashboardActivity : menu principal de l'administrateur.
- AddMovieActivity : formulaire d'ajout d'un film.
- EditMovieActivity : modification et suppression d'un film existant.
- ManageUsersActivity : affichage et suppression des utilisateurs.

4.2 Les modèles et les listes

Les classes Movie et User représentent les données manipulées dans l'application. Les classes LoginResponse, MoviesResponse, MovieResponse et UsersResponse servent à recevoir directement les réponses JSON envoyées par le serveur. Cette méthode évite d'écrire un traitement manuel des réponses.

L'affichage des collections repose sur RecyclerView avec deux adaptateurs : MovieAdapter pour les films et UserAdapter pour les utilisateurs. Le fonctionnement observé est cohérent avec une application Android de niveau débutant ou intermédiaire.

4.3 La gestion de session

La classe SessionManager utilise SharedPreferences afin de mémoriser l'état de connexion, l'identifiant, le nom d'utilisateur, l'adresse e-mail et le rôle. Après une connexion réussie, la méthode redirectByRole redirige l'utilisateur vers l'espace administrateur ou vers l'espace utilisateur.

5. Fonctionnement de l'API côté Android

La communication réseau est assurée par Retrofit. Dans ce projet, deux fichiers jouent un rôle central : RetrofitClient.java et ApiService.java.

5.1 Adresse du serveur

Le fichier RetrofitClient.java contient l'adresse de base du serveur. Toutes les routes appelées par l'application sont construites à partir de cette adresse.

```
private static final String BASE_URL = "https://fatihi.site/api/";
```

5.2 Déclaration des routes

Le fichier ApiService.java regroupe les routes disponibles. Les opérations de lecture utilisent surtout GET, alors que l'envoi de données utilise POST avec un body JSON.

Route	Méthode	Utilité
login.php	POST	Vérifier les identifiants et renvoyer les informations du compte.
register.php	POST	Créer un nouvel utilisateur avec le rôle user.
get_movies.php	GET	Récupérer la liste de tous les films.
get_movie.php	GET	Récupérer un film précis à partir de son id.
add_movie.php	POST	Ajouter un film dans la base.
update_movie.php	POST	Modifier un film existant.
delete_movie.php	POST	Supprimer un film.
get_users.php	GET	Récupérer la liste des utilisateurs.
delete_user.php	POST	Supprimer un utilisateur.

5.3 Exemple de données envoyées

Quand l'utilisateur se connecte, Android place les informations dans une structure de type Map. Par exemple, la méthode `body.put("username", username)` signifie que l'application ajoute le champ username au message qui sera envoyé au serveur.

```
Map<String, String> body = new HashMap<>();
body.put("username", username);
body.put("password", password);
```

6. Backend PHP et connexion à la base de données

Le dossier `php_api` contient la partie serveur du projet. Chaque script a une tâche précise : lire les données reçues, effectuer l'opération SQL demandée, puis renvoyer une réponse JSON.

6.1 Connexion MySQL

La connexion à la base est centralisée dans config.php. Ce fichier crée un objet PDO qui sera ensuite utilisé par tous les autres scripts PHP.

```
$pdo = new PDO(
    "mysql:host=$host;dbname=$dbname;charset=utf8mb4",
    $username,
    $password
);
```

L'utilisation de PDO est un bon choix pour ce type de projet, car elle permet d'utiliser des requêtes préparées et de mieux séparer le code PHP des requêtes SQL.

6.2 Scripts observés dans le backend

Le backend contient les scripts suivants : login.php, register.php, get_movies.php, get_movie.php, add_movie.php, update_movie.php, delete_movie.php, get_users.php et delete_user.php. Le principe est le même dans tous les cas : recevoir la requête, travailler avec la base, puis renvoyer une réponse structurée.

6.3 Tables de données déduites

Même si le script de création de la base n'est pas présent dans l'archive, il est possible de reconstituer la structure générale des tables à partir du code.

Table probable	Champs repérés dans le code
users	id, username, email, password, role, created_at
movies	id, title, description, genre, image_url, release_year, trailer_url, created_at

7. Scénarios de fonctionnement

7.1 Connexion d'un utilisateur

Dans LoginActivity, l'utilisateur saisit son nom et son mot de passe. Ces valeurs sont envoyées à login.php. Le script PHP cherche ensuite l'utilisateur dans la table users, vérifie le mot de passe avec password_verify, puis renvoie un objet JSON indiquant si la connexion a réussi ou non. Enfin, Android enregistre les informations de session et ouvre le bon écran.

7.2 Consultation de la liste des films

Dans UserHomeActivity, la méthode loadMovies appelle get_movies.php. Le script exécute une requête SELECT sur la table movies puis renvoie la liste complète. Cette liste est transformée en objets Movie et affichée dans un RecyclerView.

7.3 Ajout, modification et suppression d'un film

L'administrateur peut ouvrir AddMovieActivity pour ajouter une nouvelle fiche. Les champs remplis dans le formulaire sont envoyés à add_movie.php, qui effectue ensuite un INSERT dans la base.

Pour la modification, EditMovieActivity récupère d'abord les données du film sélectionné, puis envoie les nouvelles valeurs à update_movie.php. La suppression passe par delete_movie.php, qui supprime l'entrée correspondant à l'identifiant reçu.

7.4 Gestion des utilisateurs

L'écran ManageUsersActivity appelle get_users.php pour afficher la liste des comptes enregistrés. Lorsqu'un administrateur choisit de supprimer un utilisateur, l'application envoie son id à delete_user.php.

8. Observations sur le projet

8.1 Points positifs

- Le découpage entre Android, API et base de données est clair.
- Retrofit simplifie bien la communication HTTP.
- Le projet utilise des requêtes préparées PDO dans plusieurs scripts.
- Le mot de passe est haché à l'inscription puis vérifié à la connexion.
- L'organisation du code Android reste compréhensible pour un projet d'apprentissage.

8.2 Limites observées

- Les identifiants de connexion MySQL apparaissent dans config.php, ce qui n'est pas conseillé pour une mise en production.
- La gestion des favoris est seulement visuelle dans l'application et ne semble pas enregistrée en base de données.
- Le projet ne montre pas de base locale Android de type Room ou SQLite ; toutes les données importantes viennent du serveur.
- Certaines actions sensibles sont accessibles par des scripts PHP dédiés, ce qui demanderait une protection serveur plus complète dans un contexte réel.

9. Conclusion

Pour conclure, MovieApp est un projet cohérent pour comprendre le dialogue entre une application Android et un backend PHP/MySQL. La structure générale est simple, mais elle permet déjà de traiter plusieurs besoins concrets : authentification, consultation de données, opérations CRUD sur les films et gestion des utilisateurs.

Ce projet montre surtout que l'application mobile ne travaille pas seule. Elle agit comme une interface cliente, tandis que l'API joue le rôle d'intermédiaire avec la base de données. Du point de vue pédagogique, c'est un bon exemple pour apprendre le rôle de Retrofit, du JSON, de PDO et de la séparation entre front-end mobile et back-end.

Annexe — Fichiers les plus importants

Fichier	Fonction
LoginActivity.java	Connexion et lancement de la session
IdentificationActivity.java	Inscription d'un nouvel utilisateur
UserHomeActivity.java	Liste des films et filtres
MovieDetailActivity.java	Affichage du détail d'un film
AdminDashboardActivity.java	Navigation de l'administrateur
AddMovieActivity.java	Ajout d'un film
EditMovieActivity.java	Modification et suppression d'un film
ManageUsersActivity.java	Liste et suppression des utilisateurs
ApiService.java	Déclaration des routes API
RetrofitClient.java	Initialisation de Retrofit
config.php	Connexion PDO à MySQL